

SemOpt

LLM-Driven Code Optimization via Rule-Based Analysis

Yuwei Zhao, Yuan-An Xiao, Qianyu Xiao,
Zhao Zhang, and Yingfei Xiong

ACM TOSEM, 2026

Advisor paper discussion
6 October 2026

The optimization opportunity problem

Which strategy applies, and where?

Approach	Useful capability	Main difficulty
Direct prompting	Generates edits from the function	May overlook a relevant opportunity
Retrieve past edits	Supplies concrete examples	Code similarity can miss a shared strategy
Handwritten rules	Finds known code patterns	Requires manual work and has limited coverage

SemOpt learns reusable rules that identify where a strategy may apply.

Example: reordering condition checks

Original order

```
if costly_check(x) and x.enabled:  
    process(x)
```

Candidate order

```
if x.enabled and costly_check(x):  
    process(x)
```

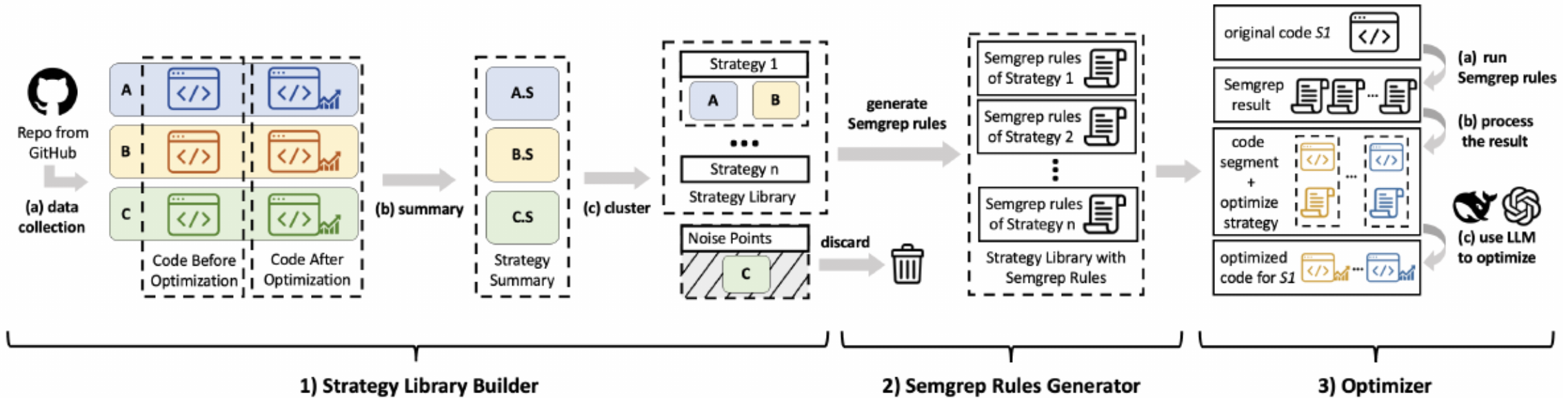
Potential benefit

When `x.enabled` is false, the candidate skips `costly_check(x)`.

Required reasoning

Skipping or reordering the call must preserve side effects and exceptions. The input distribution determines the benefit.

SemOpt architecture



- 1 Mine and group strategies
- 2 Generate rules that locate opportunities
- 3 Guide the LLM with a location and strategy

Building the reusable library

Library	Strategy clusters	Generated rules
Python	55	2,097
C/C++	155	5,057

Strategy construction

Filter commits, summarize their optimization, then cluster similar summaries.

Rule construction

Sample up to 10 commits per cluster and make 5 generation attempts per commit. Repair rules until they execute.

Selecting a location and strategy

- 1 Scan the function**
Each match identifies a location and a possible strategy.
- 2 Combine repeated matches**
Count rules supporting the same location-strategy pair.
- 3 Keep the top 25 proposals**
Use the full function, location, and strategy to prompt the LLM.

Rule support is a ranking signal. Validation still determines whether to keep an edit.

The paper uses two evaluation regimes

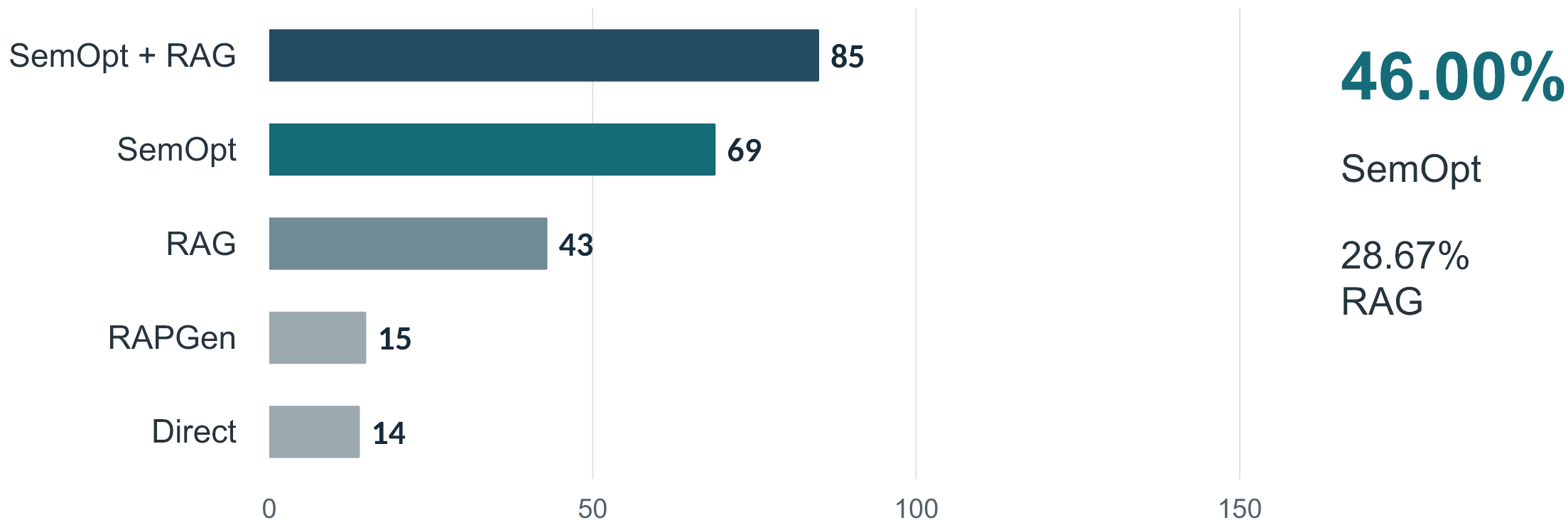
	Historical edits	Project performance
Population	151 C/C++ tasks 150 Python tasks	5 C/C++ projects 5 Python projects
Outcome	At least one candidate matches the developer edit	Tests pass and measured performance improves
Interpretation	Recovery of reference-like edits	Benefit on the measured project workloads

Python benchmark: DeepSeek-V3 only.

Ablations and other diagnostic studies: C/C++ only.

Python results: 69 of 150 tasks

Tasks with at least one reference-equivalent candidate, DeepSeek-V3

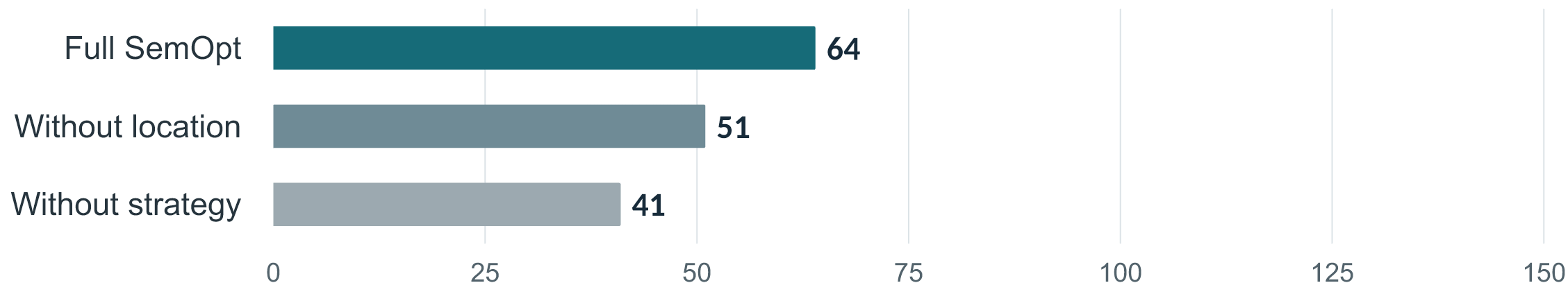


+17.33 percentage points over RAG

[1] pp. 18, 21, 24. SemOpt + RAG combines outputs and uses additional candidates.

Evidence for location and strategy guidance

SemEqv task successes on 151 C/C++ tasks



Removing examples from the same repository

Method	Standard library	Exclude same repository
SemOpt	64 successes	63 successes
RAG	36 successes	10 successes

Performance on five Python projects

Project	Tests	Mean improvement	Maximum
Click	11	143.25%	479.90%
Flask	8	56.67%	66.07%
Jinja	8	48.60%	71.43%
Requests	9	40.19%	85.45%
Scrapy	4	50.26%	61.77%

For runtime tests: improvement = (old time - new time) / new time

100% improvement means 2x speed, or a 50% reduction in runtime.

SemOpt already has an acceptance rule

1 Check behavior

Run the project's unit tests before judging performance.

2 Measure both versions

Run 6 times, discard the first run, and average the remaining 5.

More than 5% improvement on at least one test
At most 2% degradation on every other test

3 Select and compose

Keep the qualifying variant with the highest summed improvement for each function. Retain the original when none qualifies.

Critical assessment

What supports the contribution

A concrete, reusable optimization mechanism

Ablations support both prompt inputs

Repository exclusion tests transfer

Project tests show measured benefits

What bounds the conclusion

Reference patches restrict “success”

Python uses one evaluated LLM

Available tests bound correctness evidence

No described independent workload audit

The authors explicitly retain human review before adoption.

A proposed question for our PhD

How often do accepted Python edits fail independent correctness or performance checks?

Unit of study

An original-candidate pair under a declared behavior contract, workload, environment, and improvement threshold.

Hypothesis to test

Selection tests may miss harmful edits. Independent evidence may change an accept or reject decision.

Novelty requires a wider literature review and evidence that the problem matters.

A controlled first pilot

Same candidate pool for every rule

Start with 20-30 Python functions across several repositories as a feasibility batch.

Independent workloads

Separate generation/tuning inputs, decision inputs, and final audit inputs.

Decision rules to compare

SemOpt-style gate, the same gate with more measurements, and a confidence-interval gate that can defer a decision.

Outcomes

Behavioral failures and regressions, unconfirmed benefits, missed useful edits, unresolved measurements, and decision cost.

Advisor discussion

Proposed next step: audit existing decisions before designing a new acceptance method.

Initial scope

Python functions first, followed by project-level and iterative validation if the pilot justifies it.

Evidence needed to continue

Reproducible harmful approvals or missed benefits, with an independent audit and a fair measurement budget.

If the audit finds little room for improvement, revise the research direction.

Backup: metric definitions

Metric	Meaning	Reported example
Exact match	Normalized code equals the developer edit	Python 38/150
SemEqv	At least one reference-equivalent candidate	Python 69/150
Applicability	Manual judgment of a useful valid suggestion	C/C++ 89.86%
Test passing	Candidate passes the supplied project tests	Python 9,483/9,483

Click maximum: 479.90% improvement = 5.799x speed

Equivalent runtime reduction: about 82.76% on that particular test.

Backup: rule coverage and precision

Locator output	Precision	Recall
Raw matches	1.56%	82.78%
Top 25 proposals	5.13%	58.94%

Coverage argument

Under independent detection, coverage = $1 - \text{product}(1 - p_i)$.
 p_i is the detection probability of rule i .

Assumption to question

Shared models and prompts can create correlated errors. Match counts do not supply calibrated correctness confidence.

Backup: prior work and replication questions

Paper	Safeguards already studied
PIE [2]	Test-based selection and reproducible performance labels
COFFE journal study [3]	Stressful inputs and input-scale variation
SemOpt [1]	Repeated project timing and a regression threshold

Replication questions

Confirm repair limits, pilot/task denominators, candidate accounting, and equal evaluation budgets.

Keep these questions distinct from any claim that the reported results are wrong.

References and source versions

[1] Zhao et al. SemOpt: LLM-Driven Code Optimization via Rule-Based Analysis. ACM TOSEM, 2026.

<https://doi.org/10.1145/3820167>

Author PDF: xiongyingfei.github.io/papers/TOSEM26.pdf (47 pages)

[2] Shypula et al. Learning Performance-Improving Code Edits. ICLR, 2024.

<https://arxiv.org/pdf/2302.07867v5>

Published-version manuscript, arXiv v5 (25 pages)

[3] Peng et al. On the Time Efficiency of Code Generated and Optimized by Large Language Models. IEEE TSE, 2026.

<https://doi.org/10.1109/TSE.2026.3709872>

Author PDF: timm.fyi/26coffe.pdf (23 pages)

SemOpt replication package

<https://figshare.com/s/0391db49a3620ed53a1f>